

Reporte de Auditoría de Seguridad

Sistema de Monitoreo Biométrico — OficinaEficiencia VMS B2B

Campo	Detalle
Fecha	2026-05-16
Auditor	Análisis automatizado — OWASP Top 10 2021
Versión del sistema	OficinaEficiencia VMS B2B (rama `main`)
Alcance	`src/`, `config/`, `tests/`
Metodología	OWASP Top 10 2021 · Security Best Practices · Revisión estática de código
Estado	Todos los hallazgos críticos y altos **corregidos y verificados**

Resumen Ejecutivo

Se realizó una auditoría de seguridad completa del sistema biométrico de monitoreo de oficinas, cubriendo la capa de almacenamiento (SQLite), el sistema de reconocimiento facial (face_recognition / dlib), el módulo DRM de licenciamiento RSA-2048, la arquitectura multi-tenant B2B y los módulos de GUI. Se identificaron **6 vulnerabilidades** distribuidas en 3 niveles de riesgo. Todas fueron corregidas en el commit b0f77d1e.

Distribución de Hallazgos

Nivel	Cantidad	Estado
Crítico	1	Corregido
Alto	2	Corregido
Medio	3	Corregido
Bajo / Informativo	3	Aceptados / Documentados
Total	**9**	

Hallazgos Detallados

F-01 — Deserialización Insegura (Pickle)

Campo	Valor
Severidad	CRÍTICO

Campo	Valor
OWASP 2021	A08 — Software and Data Integrity Failures
CWE	CWE-502: Deserialization of Untrusted Data
Archivo	`src/recognition/face_recognizer.py`
Método	`load_known_faces()`, `save_encodings()`

Descripción

El sistema almacenaba las codificaciones biométricas de los empleados en un archivo `encodings.pkl` usando `pickle.dump()` y las cargaba con `pickle.load()`. El formato Pickle de Python es capaz de ejecutar **código arbitrario** durante la deserialización. Un atacante con acceso de escritura al directorio `%APPDATA%\OficinaEficiencia\data\faces\` (p. ej., otro proceso del sistema, malware con privilegios de usuario, o un ataque de escalada local) podría reemplazar el archivo con un payload malicioso. En el siguiente arranque del sistema, el reconocedor facial cargaría y ejecutaría ese código con los privilegios de la aplicación.

```
# ANTES – vulnerable
with open(self.encodings_file, 'rb') as f:
    data = pickle.load(f) # Ejecuta código arbitrario del archivo
```

Impacto

- Ejecución remota de código (RCE) con privilegios del proceso.
- Compromiso de todos los datos biométricos almacenados.
- Posible persistencia de malware en el arranque.

Corrección Aplicada

Se eliminó completamente el uso de Pickle. Las codificaciones se almacenan ahora en formato NumPy comprimido (`.npz`, `allow_pickle=False`), que no puede contener código ejecutable. Adicionalmente, se firma el archivo con HMAC-SHA256 usando una clave derivada del hardware de la máquina, lo que impide tanto la manipulación como el trasplante de archivos desde otra máquina.

```
# DESPUÉS – seguro
def save_encodings(self):
    buf = io.BytesIO()
    np.savez_compressed(buf, encodings=encodings_arr, names=names_arr)
    payload = buf.getvalue()
    sig = _sign(payload) # HMAC-SHA256 vinculado al hardware
    with open(self.encodings_file, 'wb') as f:
        f.write(sig + payload) # [32 bytes HMAC] + [npz payload]

def load_known_faces(self):
    raw = open(self.encodings_file, 'rb').read()
    sig, payload = raw[:32], raw[32:]
    if not _verify(payload, sig): # Rechaza si fue modificado
        raise ValueError("HMAC integrity check failed")
    loaded = np.load(io.BytesIO(payload), allow_pickle=False) # Sin pickle
```

Verificación

- Test `test_encodings_file_is_not_pickle`: verifica que el archivo no comience con opcodes Pickle.
- Test `test_tampered_file_is_rejected`: un byte modificado en el payload es detectado y rechazado.
- Test `test_foreign_machine_file_is_rejected`: un archivo firmado con clave de otra máquina es rechazado.
- Test `test_valid_roundtrip_load_save`: los datos persisten con exactitud floating-point.

F-02 — Path Traversal en Tenant ID

Campo	Valor
Severidad	ALTO
OWASP 2021	A01 — Broken Access Control
CWE	CWE-22: Improper Limitation of a Pathname
Archivo	`config/path_utils.py`
Método	`set_active_tenant()`, `get_tenant_path()`

Descripción

El `tenant_id` se almacenaba sin validación en `ConfigManager._active_tenant_id` y luego se usaba directamente en `os.path.join()` para construir rutas de sistema de archivos. Un `tenant_id` malicioso como `../../Windows/System32` o `../otroTenant` habría permitido leer o escribir archivos fuera del directorio de tenants autorizado.

```
# ANTES – vulnerable
@classmethod
def set_active_tenant(cls, tenant_id: str):
    cls._active_tenant_id = tenant_id # Sin validación

# En get_tenant_path():
tenant_base = cls.get_appdata_path("Tenants", cls.get_active_tenant())
# Con tenant_id = "../../Windows", resulta en una ruta fuera del sandbox
```

Impacto

- Aislamiento de tenants B2B comprometido: un tenant podría acceder a datos de otro.
- Escritura arbitraria de archivos en directorios del sistema.
- Lectura de archivos de licencia o configuración de otros tenants.

Corrección Aplicada

Se agregó validación con whitelist estricta (regex) antes de asignar el `tenant_id`.

```
# DESPUÉS – seguro
_TENANT_ID_RE = re.compile(r'^[A-Za-z0-9_.\-]{1,64}$')

@classmethod
def set_active_tenant(cls, tenant_id: str):
    if not isinstance(tenant_id, str) or not _TENANT_ID_RE.match(tenant_id):
        raise ValueError(
            "Invalid tenant ID: must be 1-64 characters "
            "(alphanumeric, underscore, hyphen, dot).")
    cls._active_tenant_id = tenant_id
```

La expresión regular permite únicamente caracteres alfanuméricos, guion, guion bajo y punto, con longitud de 1 a 64 caracteres. Cualquier separador de ruta (`/`, `\`) o secuencia `..` es rechazada.

Verificación

- `test_path_traversal_dot_dot_rejected`: `../../etc/passwd` lanza `ValueError`.

- `test_path_traversal_backslash_rejected`: `..\Windows\System32` lanza `ValueError`.
- `test_slash_in_tenant_id_rejected`: `valid/injected` lanza `ValueError`.
- `test_empty_tenant_id_rejected`: cadena vacía lanza `ValueError`.
- `test_valid_tenant_id_accepted`: `Empresa_ABC-01` se acepta correctamente.

F-03 — Integridad de Datos Biométricos entre Máquinas

Campo	Valor
Severidad	ALTO
OWASP 2021	A02 — Cryptographic Failures
CWE	CWE-345: Insufficient Verification of Data Authenticity
Archivo	<code>`src/recognition/face_recognizer.py`</code>
Método	<code>`load_known_faces()`</code>

Descripción

Incluso sin pickle, un archivo de encodings generado en máquina A podía ser copiado y cargado en máquina B. En un escenario de suplantación de identidad, un atacante podría generar encodings de su propio rostro en otra máquina y reemplazar el archivo legítimo, logrando que el sistema lo reconociera como un empleado registrado.

Corrección Aplicada

La firma HMAC-SHA256 incluida en F-01 resuelve también este vector. La clave del HMAC se deriva de `platform.node()` (hostname de la máquina) concatenado con una constante de dominio:

```
def _get_integrity_key() -> bytes:
    node = platform.node().encode('utf-8', errors='replace')
    return hashlib.sha256(node + b"_oe_face_integrity_v1").digest()
```

Un archivo firmado en máquina A tiene una firma inválida en máquina B, por lo que es rechazado y se dispara una re-codificación desde las imágenes originales.

F-04 — Excepciones Silenciosas en DRM

Campo	Valor
Severidad	MEDIO
OWASP 2021	A09 — Security Logging and Monitoring Failures
CWE	CWE-390: Detection of Error Condition Without Action
Archivo	<code>`src/security/drm.py`</code>
Métodos	<code>`validate_license()`</code> , <code>`_verify_and_extract()`</code>

Descripción

Los bloques `except` capturaban todas las excepciones (`Exception`) sin registrar ninguna información sobre el fallo, haciendo imposible detectar intentos de manipulación de licencias, errores de corrupción o ataques de fuerza bruta.

```
# ANTES – sin trazabilidad
except Exception:
    return False # Imposible distinguir: ¿corrupción? ¿ataque? ¿bug?
```

Corrección Aplicada

```
# DESPUÉS – con logging de seguridad
except Exception as exc:
    _logger.warning("[DRM] License validation failed: %s", type(exc).__name__)
    return False
```

Se registra el tipo de excepción (sin exponer datos sensibles como el machine ID o el contenido de la licencia) a nivel WARNING, permitiendo correlación de eventos en sistemas de monitoreo.

F-05 — Algoritmo MD5 en Pseudonimización

Campo	Valor
Severidad	MEDIO
OWASP 2021	A02 — Cryptographic Failures
CWE	CWE-327: Use of a Broken or Risky Cryptographic Algorithm
Archivo	`src/storage/database_manager.py`
Método	`anonymize_employee()`

Descripción

La función de "derecho al olvido" generaba el pseudónimo del empleado eliminado usando MD5, un algoritmo criptográficamente roto desde 2004 con colisiones conocidas. Aunque en este caso se usaba para generación de ID (no para seguridad), el uso de MD5 es una práctica insegura que podría facilitar ataques de reversión del pseudónimo.

```
# ANTES
anon_id = f"empleado_eliminado_{hashlib.md5(raw).hexdigest()[:8]}"
```

Corrección Aplicada

```
# DESPUÉS
anon_id = f"empleado_eliminado_{hashlib.sha256(raw).hexdigest()[:12]}"
```

Se usa SHA-256 y se amplía el sufijo de 8 a 12 caracteres hexadecimales para reducir la probabilidad de colisiones.

F-06 — Rutas Internas Expuestas en Stdout

Campo	Valor
Severidad	MEDIO
OWASP 2021	A09 — Security Logging and Monitoring Failures
CWE	CWE-209: Generation of Error Message Containing Sensitive Information

Campo	Valor
Archivo	`src/recognition/face_recognizer.py`

Descripción

Múltiples llamadas `print()` exponían rutas absolutas del sistema de archivos y mensajes de excepción en `stdout`, visible en consolas de depuración y potencialmente en logs del sistema operativo:

```
print(f>Loading encodings from {self.encodings_file}...) # Ruta absoluta
print(f>Error loading encodings file: {e}. Re-encoding.) # Mensaje de excepción
print(f>Error processing burst image {idx}: {e})         # Idem
```

Corrección Aplicada

Todas las llamadas `print()` fueron reemplazadas por llamadas al módulo `logging` estándar, usando `type(e).__name__` en lugar del mensaje completo de la excepción:

```
_logger = logging.getLogger(__name__)

_logger.info("[FaceRecognizer] Loaded %d face(s).", len(self.known_face_names))
_logger.warning("[FaceRecognizer] Could not load encodings (%s). Re-encoding.", type(e).__name__)
```

Hallazgos Informativos (Aceptados)

ID	Descripción	Archivo	Decisión
I-01	`_HARDWARE_SALT` hardcodeado en fuente	`drm.py`	Aceptado — PyArmor lo ofuscará en producción (Sprint 4, TASK-4.4). Riesgo residual bajo en builds de desarrollo.
I-02	Race condition TOCTOU en `check_reload()`	`face_recognizer.py`	Aceptado — ventana de riesgo mínima en entorno desktop local. No explotable remotamente.
I-03	`sys.path.append` a nivel de módulo	`zone_editor.py`	Informativo — no es una vulnerabilidad de seguridad; puede provocar conflictos de módulos.

Hallazgo Previo — SQL Injection (Resuelto en Sprint Anterior)

Campo	Valor
Severidad	ALTO (resuelto)
OWASP 2021	A03 — Injection
Archivo	`src/storage/database_manager.py`

Todos los 14 métodos de DatabaseManager usan queries parametrizadas con ?. El método anonymize_employee() usa la constante de módulo _EMPLOYEE_NAME_TABLES (hardcodeada, nunca proviene de input del usuario) para evitar inyección en el nombre de tabla del f-string. Verificado con 3 tests de regresión de inyección SQL.

Tests de Seguridad

Se entregaron **13 tests de regresión** en tests/test_security_fixes.py y **3 tests de inyección SQL** en tests/test_database_manager.py. Todos pasan en el entorno de CI.

Clase de Test	Tests	Cobertura
`TestFaceEncodingsIntegrity`	4	F-01, F-03
`TestTenantIdValidation`	7	F-02
`TestAnonymizeUsesSHA256`	2	F-05
SQL Injection (database_manager)	3	A03
Total	**16**	

===== 36 passed in 0.89s =====

Checklist OWASP Top 10 2021

OWASP	Categoría	Estado
A01	Broken Access Control	Corregido (F-02)
A02	Cryptographic Failures	Corregido (F-03, F-05)
A03	Injection	Corregido (Sprint anterior)
A04	Insecure Design	Sin hallazgos
A05	Security Misconfiguration	Informativo (I-01)
A06	Vulnerable and Outdated Components	No auditado (fuera de alcance)
A07	Identification and Authentication Failures	Sin hallazgos en alcance
A08	Software and Data Integrity Failures	Corregido (F-01)
A09	Security Logging and Monitoring Failures	Corregido (F-04, F-06)
A10	Server-Side Request Forgery	No aplica (app desktop)

Recomendaciones Pendientes

1. **Cifrado en reposo de datos biométricos** (GDPR Art. 32): los encodings están protegidos por integridad (HMAC) pero no por confidencialidad. Se recomienda cifrar el archivo `.npz` con una clave derivada del hardware (Fernet/AES-256) en una iteración futura.
2. **Auditoría de dependencias** (A06): ejecutar `pip-audit` o `safety check` contra `requirements.txt` para detectar CVEs conocidos en `face_recognition`, `dlib`, `ultralytics`, `cryptography`.
3. **Rotación de `_HARDWARE_SALT`**: planificar mecanismo de rotación del salt DRM para casos de compromiso del código fuente antes de la ofuscación con PyArmor.
4. **Sanitización de `zone_name`** en `ZoneEditor.save_zone()`: el nombre de zona se usa como clave en el JSON de zonas pero no es validado. Añadir validación de caracteres permitidos.

Reporte generado el 2026-05-16. Commit de correcciones: b0f77d1e — rama main.